Ders 13 Genişletilmiş Notu: Heap Ağaçları, Heapify, Heap'e Ekleme-Silme ve Heap Sort

Ders 13 Genişletilmiş Notu: Heap Ağaçları, Heapify, Heap’e Ekleme-Silme ve Heap Sort

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Heap Ağacı Nedir?
- 2. Heap Üzerindeki İşlemler ve Karmaşıklıkları
- 3. Heap’in Dizi Gösterimi
- 4. Heapify İşlemi
- 5. Rastgele Bir Ağaçtan Max Heap Oluşturma
- 6. Heap’in Avantajları, Dezavantajları ve Kullanım Alanları
- 7. Heap’e Ekleme (Up-Heap)
- 8. Heap’ten Silme
- 9. Heap Sort
- 10. Alıştırma Problemleri
- Örnek Sorular

1. Heap Ağacı Nedir?

Hoca, bu hafta yığın (heap) veri yapısının, gelecek hafta da öncelikli kuyrukların (priority queue) anlatılacağını söyleyerek derse girdi. Heap ağaçları, belli birkaç temel amaç etrafında günlük hayattaki pek çok problemi çözmek için kullanılır.

Derste ağırlıkla ikili yığın ağacı (binary heap tree) üzerinde duruldu. Bu ağaçta her düğümün en fazla iki çocuğu olabilir. Başka heap ağaçları da vardır, ama ders binary heap üzerinden ilerledi. Bir ağacın binary heap olabilmesi için iki kural vardır:

- **Yapı kuralı:** Ağaç tam ikili ağaç (complete binary tree) olmalıdır. Hoca bunu “olmazsa olmaz birinci özellik” diye niteledi.
- **Sıra kuralı:** Ebeveyn düğüm (parent node) her zaman çocuklarından büyük ya da her zaman çocuklarından küçük değere sahip olmalıdır.

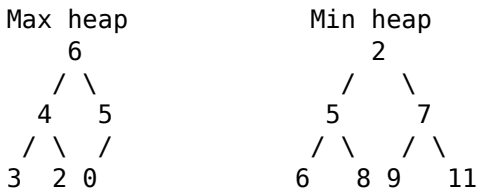
Sıra kuralı iki tür heap ağacı ortaya çıkarır:

- **Max heap:** Kökteki eleman, alt ağaçlardaki bütün düğümlerden büyüktür. Bu kural her alt ağaç için de geçerlidir.
- **Min heap:** Kökteki eleman, alt ağaçlardaki bütün düğümlerden küçüktür. Her alt ağaçta da ebeveyn çocuklarından küçüktür.

1.1 Slayttaki Örnekler

Slaytta üç ağaç vardı. Birincisi ikili ağaç özelliği gösteriyordu ama sola yaslı değildi ve son seviyede arada bir boşluk vardı. Hoca bu ağacın, düğüm değerleri ne olursa olsun heap olma ihtimalinin olmadığını söyledi; çünkü complete binary tree değildir. Yani bir ağacın heap olup olmadığına önce yapısına bakarak karar verilir. Yapı uygunsa sıra kuralı kontrol edilir.

Diğer iki ağaç şunlardı:



Max heap örneğinde kontrol şöyle yapılır: Kökün altındaki 4 ve 5, 6'dan küçük mü? Evet. Sonra 4'e bakılır: 3 ve 2, 4'ten küçük. Sonra 5'e bakılır: 0, 5'ten küçük. Demek ki bu ağaç max heap olarak kullanılabilir.

Min heap örneğinde 2'nin altındaki 5 ve 7 ondan büyüktür. 5'in altındaki 6 ve 8 büyüktür. 7'nin altındaki 9 ve 11 büyüktür.

1.2 Kardeşler Arasında Sıra Yoktur

Hoca min heap ağacında 8 ile 11'in yerini değiştirdi ve sordu: Bu ağaç yine heap olur mu?

Soru: 8 ile 11'in yeri değişince ağaç yine min heap olur mu? **Cevap:** Bir öğrenci, verilen kurallara göre olacağını söyledi. Hoca bunu doğruladı.

Hocanın dikkat çekmek istediği nokta şuydu: Düğümler kendi aralarında belli bir sıraya sahip olmak zorunda değildir. 11'in 8'den önce gelmesi "sıralama bozuldu" demek değildir. Önemli olan tek şey, ebeveyndeki değerin çocuklardaki değerlerden büyük (max heap) ya da küçük (min heap) olmasıdır. Max heap'ten söz ediyorsak bu kural bütün alt ağaçlarda geçerli olmalıdır; min heap için de aynısı.

2. Heap Üzerindeki İşlemler ve Karmaşıklıkları

2.1 Heapify Ne Demektir?

Hoca öğrencilere heapify ile neyin kastedildiğini sordu.

Soru: Heapify işlemi ne anlama gelir? **Cevap:** Bir öğrenci, ağacı az önceki kurallara uydurmak, yani en üstteki düğüm en küçük ya da en büyük olacak şekilde sıralamak ve ağacı sola yaslı hale getirmek olduğunu söyledi. Hoca doğru dedi, bir düzeltme yaptı: Ağacın bize ilk etapta kurallara uygun, yani complete binary tree olarak verildiğini kabul ediyoruz.

Ağacın yapısı bazı işlemlerden sonra, daha önce gördüğümüz ağaçlarda olduğu gibi, bozulabilir. Bu işlemler ekleme (insertion) ve silme (deletion) işlemleridir. Bir eleman eklediğinizde, onu nereye eklediğinize bağlı olarak ağacın heap yapısını bozabilirsiniz. Bir eleman sildiğinizde de aynı risk vardır. Bu yüzden her ekleme ve silmeden sonra mutlaka heapify işleminden geçilir; bir şey bozulduysa bozulan kısım heapify ile düzeltilir.

Hoca slayttaki tanımlardan birini de okudu: Heapify, aynı zamanda bir diziden heap oluşturma işlemidir (the process of creating a heap from an array). Yani heapify yalnız tek bir eleman eklendiğinde ya da silindiğinde değil, bir diziden sıfırdan heap kurmak istediğinizde de kullanılır.

2.2 Ekleme ve Silmenin Karmaşıklığı

Soru: Heap ağacında bir eleman ekleme ve silme işlemlerinin karmaşıklığı nedir? **Cevap:** Öğrenciler sohbet penceresinden $\log n$ yazdı. Hoca doğruladı ve önemli bir ekleme yaptı: Bu karmaşıklık en kötü durum (worst case) için de $O(\log n)$'dir.

Bunun sebebi ağacın complete binary tree olmasıdır. İkili arama ağacında (BST) ağaç tek tarafa yaslanıp dejenere (degenerated, pathological) bir ağaca dönüşebiliyordu ve en kötü durumda işlemler $O(n)$'e çıkıyordu. Heap'te ağacın bu şekilde bozulma ihtimali yoktur. Belli bir denge her zaman vardır. Bu yüzden ekleme ve silme en kötü durumda da $O(\log n)$'de yapılabilir.

2.3 En Büyük ve En Küçük Elemana Erişim

Soru: `getMax` ve `getMin` işlemlerinin karmaşıklığı nedir? **Cevap:** İkisi için de $O(1)$ denebilir. Ancak burada bir varsayım vardır: Max heap ile çalışıyorsak `getMax`, min heap ile çalışıyorsak `getMin` $O(1)$'dir. Çünkü aranan eleman köktedir.

Hoca ardından tersini sordu: Max heap'te en küçük elemanı ya da min heap'te en büyük elemanı bulmak istersek ne yaparız? Max heap'te en küçük eleman teknik olarak nerede olabilir?

Soru: Max heap'te en küçük eleman nerede olabilir? **Cevap:** Yapraklarda. Hoca, yaprakların yerini bildiğimizi, son seviyede olduğunu söyledi. Ağacın geri kalan seviyelerini kontrol etmek yerine yalnız yapraklarda en küçüğü (min heap için en büyüğü) aramak işi çok kısaltır.

> **Ders sonrası not:** Hoca yaprakların "son seviyede" olduğunu söyledi. Doğrusu: En küçük eleman bir yapraktadır, ama complete binary tree'de yapraklar hem son seviyede hem de sondan bir önceki seviyenin sağ tarafında olabilir. Dizi gösteriminde yapraklar $\lfloor n/2 \rfloor + 1$ ile n arasındaki gözlerdir. Yalnız bu gözlere bakmak yeter.

Bunlara bir işlem daha eklenir: `peek`. `peek` ile kökteki elemanın değeri okunur. Ağaç max heap ise en büyük, min heap ise en küçük eleman elde edilir. Özetle heap üzerinde genelde ekleme, silme ve en büyük ya da en küçük elemanın peşinde olunur. Ağacın yapısı bozulduğunda heapify ile ağaç yeniden heap özelliğine kavuşturulur.

2.4 Listede Olmayan İşlem: Arama

Hoca slayttaki işlem listesine bakıp sordu: Bugüne kadar dizilerde, bağlı listelerde hep kullandığımız temel bir işlem burada yok. Hangisi?

Soru: Heap'in işlemleri arasında hangi temel işlem yer almıyor? **Cevap:** Sohbetten gelen cevap doğruydu: arama (search).

Heap ağacı arama için uygun bir veri yapısı değildir. Elemanlar yalnız en üstte en büyük (ya da en küçük) değer olacak şekilde konumlanır. İkili arama ağacındaki gibi "küçükse sola, büyükse sağa" türünden bir kural yoktur. Bir değeri aramak için bütün ağacı gezmek gerekir. Bu yüzden arama işlemi gereken yerde heap pek tercih edilmez.

3. Heap'in Dizi Gösterimi

Heap genelde bir dizi ile tutulur. Bu derste elemanlar dizinin 1 numaralı gözünden başlar. İkili arama ağacında indeksi 0'dan başlatmıştık; burada 1'den başlatıyoruz. Böylece:

- `i` düğümünün sol çocuğu `2i` gözündedir.
- `i` düğümünün sağ çocuğu `2i + 1` gözündedir.
- `i` düğümünün ebeveyni `i / 2` (tam sayı bölmesi) gözündedir.

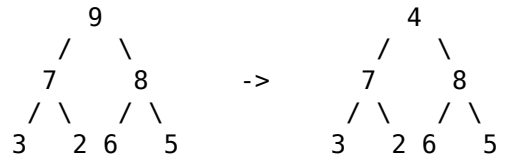
Örnek dizi: `6, 4, 5, 3, 2, 0, 1`. Bu diziyi ağaç olarak yerleştirdiğimizde yapısal olarak complete bir binary tree elde ederiz. Ebeveyndeki elemanlar çocuklardakilerden hep büyüktür. Demek ki bu yapı heap özelliği taşır; üstelik max heap'tir.

> **Hocanın vurgusu:** Size bir dizi verildiğinde üstteki elemanların büyük mü küçük mü olduğunu bilemeyebilirsiniz. Dizi max heap de olabilir, min heap de. Bu yüzden bir ağacın heap olup olmadığı sorulduğunda ikisini de kontrol etmek gerekir.

4. Heapify İşlemi

4.1 Problem: Kökteki Değer Değişti

Hoca slayttaki max heap'in kökünde 9 olduğunu, bu 9'u alıp yerine 4 koyduğunu varsaydı. Ağaç başlangıçta şuydu:



Dizi olarak `9, 7, 8, 3, 2, 6, 5` iken `4, 7, 8, 3, 2, 6, 5` oldu. Köke herhangi bir değer koyduğumuz anda ağacın yapısını değiştirmiş oluruz. Artık ağacın heap olup olmadığı mutlaka kontrol edilmelidir. Kökte 4 yerine 14 da olabilirdi; hoca bu iki durumu ayrı ayrı inceledi.

4.2 Max Heapify Kodu

Slayttaki `max heapify` fonksiyonu bu işlemi tarif eder. Fonksiyona üç bilgi gönderilir: dizi, başlangıç noktası `i` ve dizinin boyutu `n`. Sol çocuğa `2i` ile, sağ çocuğa `2i + 1` ile gidilir. Fonksiyonun kendi kendini çağırmasından, bunun özyinelemeli (recursive) bir fonksiyon olduğu anlaşılır.

```
void maxHeapify(int A[], int i, int n) {
    int left = 2 * i;
    int right = 2 * i + 1;
    int largest;
    if (left <= n && A[left] > A[i])
        largest = left;
    else
        largest = i;
    if (right <= n && A[right] > A[largest])
        largest = right;
    if (largest != i) {
        swap(&A[i], &A[largest]);
        maxHeapify(A, largest, n);
    }
}
```

Kodun adımları:

- `left <= n` ise ve sol çocuk `A[i]`'den büyükse `largest` sol çocuk olur. Değilse `largest = i` kalır.
- `right <= n` ise ve sağ çocuk `A[largest]`'tan büyükse `largest` sağ çocuk olur.
- `largest != i` ise `A[i]` ile `A[largest]` yer değiştirir ve fonksiyon bu kez `largest` gözü için kendini çağırır.

4.3 Neden İkinci Karşılaştırma `A[largest]` ile Yapılıyor?

Hoca burada bir soru sordu: İlk kontrolde `A[left] > A[i]` diye baktık. İkinci kontrolde neden `A[right] > A[i]` diye bakmıyoruz da `A[right] > A[largest]` diye bakıyoruz?

Soru: İkinci karşılaştırma neden `A[i]` ile değil, `A[largest]` ile yapılır? **Cevap:** Bir öğrenci, `largest`'ın az önce sol çocuğa atanmış olabileceğini, `A[i]` ile kontrol edersek bunu kaçıracağımızı, sağ çocuğun o elemandan da büyük olup olmadığına bakmak gerektiğini söyledi. Hoca doğruladı: Amacımız çocuklardan büyük olanı yukarı taşımaktır. O ana kadar bulunan en büyüğü `largest`'ta tutarız.

4.4 Adım Adım İz: Kökte 4

Başlangıç: `4, 7, 8, 3, 2, 6, 5`, `i = 1`.

1. `i = 1` (değer 4). Sol çocuk 2. gözde 7'dir. $7 > 4$ olduğu için `largest = 2` olur. Sağ çocuk 3. gözde 8'dir. $8 > 7$ olduğu için `largest = 3` olur. `largest != i` olduğu için 4 ile 8 yer değiştirir: `8, 7, 4, 3, 2, 6, 5`.
2. Fonksiyon `largest = 3` ile kendini çağırır. Artık `left = 6`, `right = 7`. 4, 6 ve 5 ile karşılaştırılır. $6 > 4$ olduğu için `largest = 6` olur. $5 > 6$ değildir. 4 ile 6 yer değiştirir: `8, 7, 6, 3, 2, 4, 5`.
3. Fonksiyon `i = 6` ile çağrılır. `left = 12 > n` olduğu için çocuk yoktur; 4 yaprağa inmiştir. `largest == i` kalır ve özyineleme biter.

Sonuç: `8, 7, 6, 3, 2, 4, 5`. Hoca slayttaki figürde ara durumu da gösterdi: önce 8 yukarı çıktı, sonra 4, 6, 5 üçlüsünde 6 yukarı çıktı. Yaprağa ulaşıldığında özyinelemeli çağrılar geri dönerek işi bitirdi ve 4 olması gereken konuma yerleşti.

4.5 Kökte 14 Olsaydı

Köke 4 yerine 14 konsaydı: “7, 14’ten büyük mü?” sorusunun cevabı hayır, “8, 14’ten büyük mü?” sorusunun cevabı da hayır olurdu. `largest` değişmez, `i` ile aynı kalırdı. Alt düğümlerde değişikliğe gerek olmadığı için fonksiyon kendini çağırmaz ve heapify ilk adımda biterdi. Demek ki durma koşullarından biri şudur: Alt elemanlardan biri yukarı taşınıyor mu? Taşınmıyorsa iş biter. Diğer durma koşulu yaprağa ulaşmaktır.

4.6 Heapify’ın Karmaşıklığı

Soru: Heapify’ın en fazla karmaşıklığı nedir? Yukarıdan aşağı doğru iniyoruz; kaç adımda inebiliriz? **Cevap:** Bir öğrenci $\log n$ dedi. Hoca doğruladı.

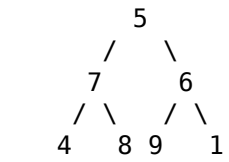
Eleman her adımda bir seviye iner. En fazla ağacın yüksekliği kadar adım atılır. Complete binary tree’nin yüksekliği $\lfloor \log_2 n \rfloor$ olduğu için heapify $O(\log n)$’dir.

5. Rastgele Bir Ağaçtan Max Heap Oluşturma

5.1 Problem

Önceki örnekte ağacın heap olduğunu biliyorduk ve yalnız bir değeri değiştirmiştik. O tek elemanı yerleştirdikten sonra gerekli kontrolleri yapmak yetiyordu. Peki ağaç hakkında hiçbir bilgimiz yoksa? Bize bir ağaç verilir ve bunun max heap’e dönüştürülmesi istenir.

Slayttaki ağaç `5, 7, 6, 4, 8, 9, 1` idi:



Soru: Bu ağacı max heap’e dönüştürmek için ne yapmalıyız? **Cevap:** Bir öğrenci, önce sol ve sağ alt ağaçları heapify yapıp sonra kökü heapify yapmayı, bir değişiklik olursa değişen tarafı tekrar heapify yapmayı önerdi. Hoca doğru dedi.

Soru: Heapify’a hangi düğümden başlamalıyız? **Cevap:** Öğrenci 2 numaralı düğümden, yani 7’den başlamayı önerdi. Hoca kritik noktanın başka olduğunu söyledi: Yaprak olmayan son iç düğüm (internal node) hangisidir? İşe oradan başlanır. Bu ağaçta o düğüm 3 numaralı düğümdür (değeri 6). Yapraklar 4, 8, 9 ve 1’dir.

Hoca bir de sıra sorusu sordu: Heapify’ı yukarıdan aşağı mı, aşağıdan yukarı mı uygulayacağız? Bir öğrenci, heapify özyinelemeli olduğuna göre tepeden başlayıp 9’a kadar inip onu yukarı almanın gerekmediğini sordu. Hoca cevapladı: Her tek heapify çağrısı aşağı yönlü çalışır. Ama iç düğümler sondan başa doğru, yani aşağıdan yukarı sırayla ele alınır. Böylece bir düğüm işlenirken alt ağaçları zaten heap olmuş olur.

5.2 Build Max Heap Kodu

Slayttaki kod bu işi yapar: `n / 2`’den, yani iç düğümlerin sonuncusundan başlanır ve 1 numaralı düğüme kadar bütün düğümler için `maxHeapify` çağrılır.

```
void buildMaxHeap(int A[], int n) {
    for (int i = n / 2; i >= 1; i--)
        maxHeapify(A, i, n);
}
```

5.3 Örnek 1: `5, 7, 6, 4, 8, 9, 1`

$n = 7$, $n/2 = 3$. Tur tur:

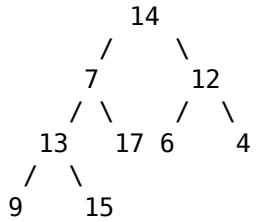
Tur	`i` (değer)	Karşılaştırma	Sonuç
1	3 (6)	Çocuklar 9 ve 1; 9 en büyük. 6 ile 9 yer değiştirir, 6 yaprağa iner	5, 7, 9, 4, 8, 6, 1
2	2 (7)	Çocuklar 4 ve 8; 8 en büyük. 7 ile 8 yer değiştirir, 7 yaprağa iner	5, 8, 9, 4, 7, 6, 1
3	1 (5)	Çocuklar 8 ve 9; 9 yukarı çıkar, 5 iner	9, 8, 5, 4, 7, 6, 1
3 (devam)	3 (5)	5 henüz yaprak değil; çocuklar 6 ve 1; 6 yukarı çıkar, 5 yaprağa iner	9, 8, 6, 4, 7, 5, 1

Üçüncü turda 5 kökten bir seviye indikten sonra tur bitmez, çünkü 5'in altında hâlâ düğümler vardır. 5 kendini 6 ve 1 ile kıyaslar, 6 ile yer değiştirir ve yaprağa ulaşır. Bütün iç düğümler işlendiği için ağaç artık max heap'tir: `9, 8, 6, 4, 7, 5, 1`.

Hoca aradan sonra aynı örneği tekrar özetledi: 6 heapify'a sokulur, 9 yukarı çıkar. 7'ye geçilir, 8 yukarı çıkar. Son olarak 5 heapify'a sokulur; 9 yukarı çıkar, 5 iner, sonra 6 yukarı çıkar ve 5 yaprağa ulaşır.

5.4 Örnek 2: `14, 7, 12, 13, 17, 6, 4, 9, 15`

Hoca tahtaya yeni bir ağaç çizdi ve düğümleri 1'den 9'a kadar numaraladı:



Gözle bakınca bu ağacın heap olmadığı görülür. Hoca, bizim gözle karar vermemize karşın algoritmanın bunu analitik olarak bulabilmesi gerektiğini söyledi; bunun da üzerinde çalışılabilecek örnek algoritmalardan biri olduğunu ekledi.

Soru: Bu ağaçta hangi iç düğümden başlarız? **Cevap:** Sohbetten gelen cevap: 4. düğüm, yani 13. Hoca doğruladı. 15, 9, 17 (5. düğüm), 6 ve 4 yapraktır. $n = 9$ için $n/2 = 4$'tür.

Tur	`i` (değer)	İşlem	Sonuç
1	4 (13)	Çocuklar 9 ve 15; 15 yukarı, 13 yaprağa iner	14, 7, 12, 15, 17, 6, 4, 9, 13
2	3 (12)	Çocuklar 6 ve 4; ikisi de küçük, değişiklik yok	14, 7, 12, 15, 17, 6, 4, 9, 13
3	2 (7)	Çocuklar 15 ve 17; 17 yukarı, 7 yaprağa iner	14, 17, 12, 15, 7, 6, 4, 9, 13
4	1 (14)	Çocuklar 17 ve 12; 17 yukarı, 14 iner	17, 14, 12, 15, 7, 6, 4, 9, 13
4 (devam)	2 (14)	Çocuklar 15 ve 7; 15 yukarı, 14 iner	17, 15, 12, 14, 7, 6, 4, 9, 13

Tur	`i` (değer)	İşlem	Sonuç
4 (devam)	4 (14)	Çocuklar 9 ve 13; ikisi de küçük, heapify biter	17, 15, 12, 14, 7, 6, 4, 9, 13

Üçüncü turda 7 aşağı inince yaprağa ulaşır. Sol taraftaki 15, 9, 13 üçgeninde değişiklik olmadığı için oraya bakmaya gerek kalmaz. Son turda 14 iki seviye iner ve 9, 13 çocuklarından büyük olduğu için orada kalır.

Hoca sonucu kontrol etti: 17'nin çocukları 15 ve 12, 15'in çocukları 14 ve 7, 12'nin çocukları 6 ve 4, 14'ün çocukları 9 ve 13. Hepsi kurala uyuyor; max heap'e dönüşüm tamamlandı. Çalışan kod `buildMaxHeap`'tir; o da her iç düğüm için `maxHeapify`'ı çağırır. `maxHeapify` her iç düğüm için yaprağa ulaşana kadar ya da eleman yer değiştirmeyene kadar sürer.

> **Ek bilgi (derste anlatılmadı):** `buildMaxHeap` içinde $n/2$ kez $O(\log n)$'lik heapify çağrıldığı için kaba üst sınır $O(n \log n)$'dir. Sıkı analiz ise $O(n)$ verir. Yüksekliği h olan bir düğüm en fazla h seviye iner ve yüksekliği h olan düğüm sayısı en fazla $\lceil n/2^{h+1} \rceil$'dir. Toplam iş $\sum_{h \geq 0} \frac{n}{2^{h+1}} \cdot h = O(n)$ olur, çünkü $\sum_{h \geq 0} h/2^h = 2$'dir.

6. Heap'in Avantajları, Dezavantajları ve Kullanım Alanları

6.1 Avantajlar

- **En büyük ve en küçük elemana hızlı erişim:** Max heap'te en büyük, min heap'te en küçük elemana $O(1)$'de erişilir.
- **Verimli ekleme ve silme:** İkisi de $O(\log n)$'de tamamlanır.
- **Kolay gerçekleme:** Dizi kullanılarak çok basit biçimde gerçeklenebilir.
- **Gerçek hayat uygulamalarına uygunluk:** Pek çok uygulamada kullanılır.

6.2 Dezavantajlar

- **Arama için uygun değildir:** Özellikle en büyük ya da en küçük değerin değil de ara değerlerin peşindeyseniz heap kullanmak akıllıca değildir.
- **Ek yük (overhead):** Heap yapısının ek bir maliyeti vardır.
- **Öncelik gerekmeyen kuyruklarda yavaştır:** Öncelik içermeyen kuyruklarda heap, dizi ya da bağlı liste yapılarına göre daha yavaş çalışır. Öncelikli kuyruk kullanmıyorsanız normal bir bağlı liste ya da dizi kullanmak çok daha mantıklıdır.

6.3 Kullanım Alanları

- **Öncelikli kuyruklar (priority queue):** Gelecek hafta görülecek. Gerçek hayatta öncelikli kuyrukların pek çok çeşidi vardır.
- **Heap sort:** Bu derste görülecek. $O(n \log n)$'de çalışır, teknik olarak merge sort ve quicksort'un rakibidir ama çeşitli handikapları yüzünden çok sık tercih edilmez.
- **Graf algoritmaları:** En kısa yol için Dijkstra, minimum yayılan ağaç (minimum spanning tree) algoritmalarından Prim ve A* (A yıldız) arama algoritması.
- **Sıkıştırma:** Özellikle dosya sıkıştırmada Huffman kodlaması.
- **Dinamik programlama:** Bazı problemlerin çözümünde.
- **Tıbbi uygulamalar:** Hastanın durumunun kritikliğine göre önce tedavi almasını sağlayan sistemlerde, öncelikle hizmet alacak hastalar en üstte konumlanır.
- **Yük dengeleme (load balancing):** Sunucular arasında yük dağıtılırken, yükü yüksek sunuculara iş verip dengeyi bozmamak için en düşük yükteki sunucuya iş vermek amacıyla min heap'ten yararlanılır.
- **Finans:** Bu alanda da heap'in kullanıldığı pek çok örnek vardır.

Hoca bu örneklerin çoğunun temelinde öncelikli kuyruk olduğunu, özellikle en küçük ve en büyük değerlere erişim söz konusu olduğunda heap'in çok yoğun kullanıldığını söyledi.

7. Heap'e Ekleme (Up-Heap)

Heap üzerindeki işlemler ağırlıkla ekleme ve silme üzerinden yürür; bir de kök değerini okuyan `peek` (`getMax`, `getMin`) vardır. Hoca ekranı ikiye bölüp bir tarafta eklemeyi, diğer tarafta silmeyi anlattı.

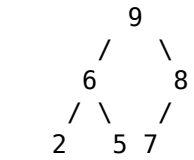
Ekleme kuralı ağacın complete binary tree kalmasına dayanır. Yeni gelen eleman, dizinin son elemanından sonraki ilk boş göze konur. Örneğin son eleman 7 ise yeni eleman 7'den sonraki göze gelir. 10'dan sonra 11 ya da 20 gelseydi, o da 10'dan sonraki ilk göze konacaktı.

Eleman sona konduğu için heap özelliği bozulabilir. Daha önce aşağı yönlü heapify (down-heap) yapıyorduk. Ekleme sonrasında ise yukarı yönlü heapify (up-heap) yapılır: Yeni eleman ebeveyniyle karşılaştırılır; ebeveyninden büyükse yer değiştirir. Bu, eleman köke ulaşana ya da kendisinden büyük bir ebeveyne gelene kadar sürer.

```
void insertHeap(int A[], int *n, int x) {
    (*n)++;
    int i = *n;
    A[i] = x;
    while (i > 1 && A[i / 2] < A[i]) {
        swap(&A[i], &A[i / 2]);
        i = i / 2;
    }
}
```

Bu kod derste yazılmadı; hocanın anlattığı adımlarla kuruldu (ara adım eklendi).

7.1 Örnek 1: `9, 6, 8, 2, 5, 7` ağacına 10 ekleme



1. 10, 7'den sonraki boş göze (7. göz) konur: `9, 6, 8, 2, 5, 7, 10`. Ebeveyni 3. gözdeki 8'dir.
2. 10 > 8 olduğu için yer değiştirir: `9, 6, 10, 2, 5, 7, 8`. 10'un yeni ebeveyni kökteki 9'dur.
3. 10 > 9 olduğu için yer değiştirir: `10, 6, 9, 2, 5, 7, 8`. 10 köke ulaştı.

Sonuç: `10, 6, 9, 2, 5, 7, 8`.

7.2 Örnek 2: `9, 8, 6, 1, 2, 5` ağacına 7 ekleme

1. 7, 5'ten sonraki göze (7. göz) konur. Ebeveyni 3. gözdeki 6'dır.
2. 7 > 6 olduğu için yer değiştirir: `9, 8, 7, 1, 2, 5, 6`.
3. 7'nin ebeveyni 9'dur. 7 > 9 değildir, 7 yerinde kalır.

Sonuç: `9, 8, 7, 1, 2, 5, 6`. Hoca burada şuna dikkat çekti: Yukarı taşınma ihtimali olan yalnız yeni gelen elemandır. Üst taraftaki 8 gibi bir elemanın yer değiştirme ihtimali yoktur; yalnız 7'nin ebeveyninden büyük olup olmadığına bakılır.

7.3 Örnek 3: aynı ağaca 12 ekleme

Ağaç: `9, 8, 7, 1, 2, 5, 6`.

1. 12, 8. göze konur. Ebeveyni 4. gözdeki 1'dir. 12 > 1, yer değiştirir: `9, 8, 7, 12, 2, 5, 6, 1`.
2. 12'nin ebeveyni 2. gözdeki 8'dir. 12 > 8, yer değiştirir: `9, 12, 7, 8, 2, 5, 6, 1`.
3. 12'nin ebeveyni kökteki 9'dur. 12 > 9, yer değiştirir: `12, 9, 7, 8, 2, 5, 6, 1`.

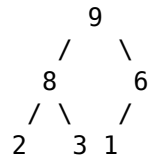
Sonuç: `12, 9, 7, 8, 2, 5, 6, 1`.

7.4 Ekleme ile Heapify Arasındaki Fark

Hoca, eklemenin az önceki aşağı yönlü heapify'dan farkına dikkat çekti: Eklemede çocuklar arasında karşılaştırma yapılmaz. Elimizdeki ağaç zaten heap'tir. Yeni gelen elemanın yalnız ebeveyniyle karşılaştırılması yeter. Üçlü bir karşılaştırmaya (ebeveyn ve iki çocuk) gerek kalmaz ve eleman bu sayede yukarı taşınır. Eleman en fazla ağacın yüksekliği kadar yukarı çıkar; bu yüzden ekleme $O(\log n)$'dir.

8. Heap'ten Silme

8.1 Örnek: `9, 8, 6, 2, 3, 1` ağacından 9'u silme



Silinmesi istenen eleman 9'dur. Çıkış noktası şudur: 9'u ağacın en son gözüne taşımak işi kolaylaştırır.

1. Silinecek eleman (9) ile en sondaki eleman (1) yer değiştirir: `1, 8, 6, 2, 3, 9`.
2. Eleman sayısı `n` bir azaltılır, 6'dan 5'e iner. 9 böylece dizide dursa da artık devre dışıdır: `1, 8, 6, 2, 3`.
3. Ağacın son elemanı yukarı taşındığı için ağaç bozuldu. Bundan sonrası tamamen heapify işidir. 1'in çocukları 8 ve 6'dır; 8 daha büyüktür. 8 yukarı çıkar, 1 aşağı iner: `8, 1, 6, 2, 3`.
4. 1 henüz yaprakta değildir. Çocukları 2 ve 3'tür; 3 daha büyüktür. 3 yukarı çıkar, 1 aşağı iner: `8, 3, 6, 2, 1`.
5. 1 yaprağa ulaştı; heapify biter.

Sonuç: `8, 3, 6, 2, 1`. Hoca, dizi üzerinde çalıştığımızı varsayınca silinen elemanın son gözle yer değiştirip `n`'in azaltılmasıyla otomatik olarak devre dışı kaldığını vurguladı.

8.2 Silme Adımları

- Silinecek elemanı bulun.
- O elemanı ağacın en sonundaki elemanla yer değiştirin (swap). En sondaki eleman silinecek elemanın yerine gelir.
- Ağaçtaki eleman sayısını bir azaltın. Sona gönderilen eleman böylece devre dışı kalır.
- Yerine yeni yerleştirilen elemandan başlayarak heapify yapın.

Heapify'ın sonunda ağaç, silme işleminden sonra olması gereken hale gelir.

```
int deleteMax(int A[], int *n) {
    int max = A[1];
    swap(&A[1], &A[*n]);
    (*n)--;
    maxHeapify(A, 1, *n);
    return max;
}
```

Bu kod derste yazılmadı; hocanın verdiği adımlarla kuruldu (ara adım eklendi).

8.3 Yalnız Kök mü Silinir?

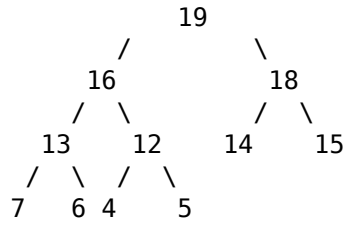
Soru: Arama işlemini maliyetli olduğu için yapmıyorduk. Silme işlemi yalnız max heap'te en büyükten, min heap'te en küçükten mi yapılıyor? **Cevap:** Yapı genellikle bu yönde kurgulanmıştır. Mantık, öncelikli kuyruktan eleman çıkarmaktır: En üstteki, en öncelikli eleman (değere göre en büyük ya da en küçük) çıkarılır. Bu örnekteki silme de bunu tarif etti. Ama silme mutlaka böyle olmak zorunda değildir; ağacın herhangi bir elemanını çıkarmak her zaman mümkündür. Yine de silme ağırlıkla en tepedeki elemanı çıkarmak için yapılır, çünkü asıl amaç öncelikli elemanın kuyruktan çıkarılmasıdır.

Ders sonrası not: Kök dışında bir eleman silinirken yerine gelen son eleman, yeni yerinin ebeveyninden büyük olabilir. Bu durumda aşağı yönlü heapify değil, eklemedeki gibi yukarı yönlü düzeltme (up-heap) gerekir. "Yerine gelen elemandan heapify" kuralı kök silmede tam doğrudur; ortadan silmede eleman önce ebeveyniyle karşılaştırılmalıdır.

9. Heap Sort

9.1 Heap Sıralı Değildir

Hoca tahtaya şu max heap'i çizdi:



Bu ağacı dizi olarak yazdırdığınızda `19, 16, 18, 13, 12, 14, 15, 7, 6, 4, 5` görürsünüz. Nispeten sıralı denebilir ama aslında sıralı değildir.

Hocanın vurgusu: "Heap ağacı en üstte en büyük elemanı tutuyor, altındakiler ondan küçük olduğu için zaten sıralıdır" mantığı doğru değildir. Dizinin yazdırılmış haline bakınca bu rahatça görülür.

9.2 Fikir Yürütme: Heap'ten Sıralama

Hoca, bir sonraki slaytta ayrıntılı anlatım olsa da "beyin jimnastiği" olsun diye sordu: En tepede en büyük elemanın olduğu bu max heap'ten yararlanarak elemanları nasıl sıralarsınız?

Soru: Max heap'i kullanarak elemanları nasıl sıralarız? **Cevap:** Öğrencilerden üç fikir geldi:

- Bir öğrenci, ilk elemanı sırayla görmezden gelip kalanı yeniden heap'e çevirmeyi önerdi; uygulamasından emin değildi. Hoca, başlangıç olarak böyle bir adımın zaten gerektiğini söyledi.
- Bir öğrenci, her seviyedeki elemanları soldan sağa sıralamayı önerdi: En büyükler yukarıda olduğuna göre her seviye sıralanırsa bütün liste sıralanır. Hoca mantıken yapılabileceğini ama maliyetli olduğunu söyledi. Ağaç büyüdükçe her seviyedeki eleman sayısı n mertebesine gider ve bunları sıralamak için belki n^2 karmaşıklığında bir algoritma ya da başka sıralama algoritmaları gerekir. Amaç heap'in kendi yapısını kullanmaktır.
- Bir öğrenci, her düğümün çocuklarını kendi arasında sıralamayı önerdi. Hoca önce "tam olarak olmaz" dedi, sonra bu örnekte olabilir gibi durduğunu ama aranan çözümün bu olmadığını söyledi.

  Ders sonrası not: Bu örnekte seviyeleri ayrı ayrı büyükten küçüğe sıralayıp art arda yazmak `19, 18, 16, 15, 14, 13, 12, 7, 6, 5, 4` verir ve sonuç gerçekten sıralıdır (python ile doğrulandı). Ama bu genel bir kural değildir. Heap yalnız ebeveyn ile çocuk arasında sıra koyar. Bir alt ağaçtaki derin bir düğüm, başka bir alt ağaçtaki daha üst seviyedeki bir düğümden büyük olabilir. Bu yüzden seviye seviye sıralama her heap'te doğru sonuç vermez.

Hoca ardından silme işlemini hatırlattı ve her turda en tepede en büyük elemanın olacağını düşünerek ilerlemeyi önerdi.

9.3 Heap Sort Mantığı

- Kökteki en büyük eleman, dizinin son elemanıyla yer değiştirir.
- `n` bir azaltılır. Son göz artık sıralı kısma geçmiştir.
- Ağaç bozulduğu için kökten heapify yapılır. Kökte yine kalan elemanların en büyüğü olur.
- Bir sonraki turda kök, bir önceki turda kullanılan gözden bir önceki gözle yer değiştirir.
- İşlem tek eleman kalana kadar tekrarlanır.

Hoca bunu “çok basit bir mantık” diye özetledi: En üstteki eleman dizinin son elemanıyla yer değiştirir. Son eleman en büyük olduğu için bir sonraki turda tepedeki eleman sondan bir önceki göze konur. Her turda bir önceki göze yerleştirilir ve her yer değiştirmeden sonra heapify yapılır.

9.4 Örnek 1: `9, 6, 8, 2, 5, 7`

Aşağıdaki tabloda | işaretinin sağı sıralı kısımdır.

Tur	Yer değiştirme	Heapify	Dizi
1	9 ↔ 7	7’nin çocukları 6 ve 8; 8 yukarı, 7 iner	8, 6, 7, 2, 5 | 9
2	8 ↔ 5	5’in çocukları 6 ve 7; 7 yukarı, 5 iner	7, 6, 5, 2 | 8, 9
3	7 ↔ 2	2’nin çocukları 6 ve 5; 6 yukarı, 2 iner	6, 2, 5 | 7, 8, 9
4	6 ↔ 5	5’in çocuğu 2; değişiklik yok	5, 2 | 6, 7, 8, 9
5	5 ↔ 2	Tek eleman kaldı	2 | 5, 6, 7, 8, 9

Sonuç: `2, 5, 6, 7, 8, 9`.

> **Ders sonrası not:** Hoca birinci turun sonunu “8, 7, 6, 2, 5, 9” diye okudu. 7 ile 8 yer değiştirdiği için doğru dizi `8, 6, 7, 2, 5 | 9`’dur. Hocanın anlattığı karşılaştırma (7, 6, 8’den 8 yukarı çıktı) bu sonuçla tutarlıdır; yalnız okuma sırası karışmıştır.

9.5 Örnek 2: `19, 16, 18, 13, 12, 14, 15, 7, 6, 4, 5`

Hoca aynı mantığı tahtadaki büyük örnek üzerinde de uyguladı. Her turdaki heapify takasları ve sonuç:

Tur	Yer değiştirme	Heapify’da yukarı çıkanlar	Dizi
1	19 ↔ 5	18, sonra 15	18, 16, 15, 13, 12, 14, 5, 7, 6, 4 | 19
2	18 ↔ 4	16, sonra 13, sonra 7	16, 13, 15, 7, 12, 14, 5, 4, 6 | 18, 19
3	16 ↔ 6	15, sonra 14	15, 13, 14, 7, 12, 6, 5, 4 | 16, 18, 19
4	15 ↔ 4	14, sonra 6	14, 13, 6, 7, 12, 4, 5 | 15, 16, 18, 19
5	14 ↔ 5	13, sonra 12	13, 12, 6, 7, 5, 4 | 14, 15, 16, 18, 19
6	13 ↔ 4	12, sonra 7	12, 7, 6, 4, 5 | 13, 14, 15, 16, 18, 19
7	12 ↔ 5	7	7, 5, 6, 4 | 12, 13, 14, 15, 16, 18, 19
8	7 ↔ 4	6	6, 5, 4 | 7, 12, 13, 14, 15, 16, 18, 19
9	6 ↔ 4	5	5, 4 | 6, 7, 12, 13, 14, 15, 16, 18, 19
10	5 ↔ 4	Yok	4 | 5, 6, 7, 12, 13, 14, 15, 16, 18, 19

Tablodaki takasların hepsi hocanın tahtadaki anlatımıyla aynıdır ve python ile doğrulandı.

Hocanın bu örnekte vurguladığı nokta `n`’in her turda bir azalmasıdır. Örneğin beşinci turda 5, 12 ile yer değiştirip aşağı indiğinde, 5’in altındaki gözler artık `n`’in ötesindedir; 18 ve 19 gibi

sıralı kısımdaki elemanlarla karşılaştırma yapılmaz. 5, yaprak konumunda kabul edilir. Aynı şekilde sekizinci turda 4 indiğinde altındaki 13 ve 14 artık çocuk sayılmaz.

> **Ders sonrası not:** Hoca üçüncü turda "en son koyduğum 18'i bu sefer 6'ya alacağım" dedi. O turda kökte 16 vardır; yer değiştiren 16 ile 6'dır. Hocanın tahtadaki sonraki işlemleri de buna uygundur.

9.6 Tam Program

Ders boyunca anlatılan heapify, build max heap, ekleme, silme ve heap sort işlemlerinin hepsini dersteki örneklerle çalıştıran program aşağıdadır. Program `gcc -Wall -Wextra` ile uyarısız derlendi ve çıktısı yukarıdaki tablolarla aynıdır.

```c
#include <stdio.h>

#define MAX 100

void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}

/* A[1..n] dizisinde i düğümünü aşağı yönlü yerleştirir */
void maxHeapify(int A[], int i, int n) {
    int left = 2 * i;
    int right = 2 * i + 1;
    int largest;
    if (left <= n && A[left] > A[i])
        largest = left;
    else
        largest = i;
    if (right <= n && A[right] > A[largest])
        largest = right;
    if (largest != i) {
        swap(&A[i], &A[largest]);
        maxHeapify(A, largest, n);
    }
}

void buildMaxHeap(int A[], int n) {
    for (int i = n / 2; i >= 1; i--)
        maxHeapify(A, i, n);
}

/* Ekleme: son gözden sonra koy, ebeveynle karşılaştırarak yukarı taşı */
void insertHeap(int A[], int *n, int x) {
    (*n)++;
    int i = *n;
    A[i] = x;
    while (i > 1 && A[i / 2] < A[i]) {
        swap(&A[i], &A[i / 2]);
        i = i / 2;
    }
}

/* Kökten silme: son elemanla yer değiştir, n'i azalt, kökten heapify */
int deleteMax(int A[], int *n) {
    int max = A[1];
    swap(&A[1], &A[*n]);
    (*n)--;
```

```
    maxHeapify(A, 1, *n);
    return max;
}

void heapSort(int A[], int n) {
    buildMaxHeap(A, n);
    for (int i = n; i >= 2; i--) {
        swap(&A[1], &A[i]);
        maxHeapify(A, 1, i - 1);
    }
}

void print(const char *s, int A[], int n) {
    printf("%s", s);
    for (int i = 1; i <= n; i++)
        printf("%d ", A[i]);
    printf("\n");
}

int main(void) {
    int h[MAX] = {0, 4, 7, 8, 3, 2, 6, 5};
    maxHeapify(h, 1, 7);
    print("heapify: ", h, 7);

    int b1[MAX] = {0, 5, 7, 6, 4, 8, 9, 1};
    buildMaxHeap(b1, 7);
    print("build 1: ", b1, 7);

    int b2[MAX] = {0, 14, 7, 12, 13, 17, 6, 4, 9, 15};
    buildMaxHeap(b2, 9);
    print("build 2: ", b2, 9);

    int e[MAX] = {0, 9, 8, 6, 1, 2, 5};
    int n = 6;
    insertHeap(e, &n, 7);
    print("ekle 7:  ", e, n);
    insertHeap(e, &n, 12);
    print("ekle 12: ", e, n);

    int d[MAX] = {0, 9, 8, 6, 2, 3, 1};
    n = 6;
    int m = deleteMax(d, &n);
    printf("silinen %d, ", m);
    print("kalan: ", d, n);

    int s[MAX] = {0, 19, 16, 18, 13, 12, 14, 15, 7, 6, 4, 5};
    heapSort(s, 11);
    print("heap sort: ", s, 11);
    return 0;
}
```

Çıktı:

```
heapify: 8 7 6 3 2 4 5
build 1: 9 8 6 4 7 5 1
build 2: 17 15 12 14 7 6 4 9 13
ekle 7:  9 8 7 1 2 5 6
ekle 12: 12 9 7 8 2 5 6 1
silinen 9, kalan: 8 3 6 2 1
heap sort: 4 5 6 7 12 13 14 15 16 18 19
```

Dizilerin 0. gözü kullanılmaz; derste olduğu gibi indeks 1'den başlar. `heapSort` fonksiyonu dersteki örneklerin aksine işe `buildMaxHeap` ile başlar; böylece rastgele bir dizi de sıralanabilir. Dersteki örneklerde ağaç zaten max heap olarak verilmişti.

9.7 Karmaşıklık

Soru: Heap sort'un karmaşıklığı nedir? **Cevap:** Sohbetten önce n cevabı geldi. Hoca "bir düşünelim, burada kaç tane iş yapıyoruz?" diye sordu. Bir öğrenci $n \log n$ dedi ve gerekçesini açıkladı: Heapify işlemi $\log n$'dir ve bunu her eleman için tekrarlıyoruz, yani $n \cdot \log n$. Hoca doğruladı.

Aradan sonra hoca bunu tekrar özetledi: Heapify her eleman için $O(\log n)$'lik bir maliyet oluşturur. En kötü durumda n elemanın her biri heapify işlemine girdiği için heap sort $O(n \log n)$'dir.

9.8 Kararlılık

> **Hocanın vurgusu:** Heap sort'un temel sıkıntısı kararlı (stable) olmamasıdır.

Kararlı olmamak şu demektir: Dizide aynı değere sahip iki eleman varsa, sıralamadan sonra bunların göreli sırasının korunacağı garanti edilmez. Önce gelen, sıralamadan sonra arkaya geçebilir. Hocanın örneği aynı ada sahip iki kayıttı: Bu kayıtların hangisinin önce geldiği önemliyse heap sort bunu sağlayamaz. Bu yüzden heap sort kararlı olmayan sıralama algoritmaları arasındadır.

Hoca, heapify bir kez kodlanıp tasarlandıktan sonra diğer işlerin (ekleme, silme, sıralama) çok zor olmadığını ekledi.

9.9 Yalnız İlk k Eleman

Heap sort'un bir avantajı da k'ıncı en küçük ya da k'ıncı en büyük elemanın peşinde olunduğunda ortaya çıkar. Bütün diziyi sıralamak yerine yalnız k tur yapılırsa karmaşıklık $O(k \log n)$ düzeyine iner. Hoca, öncelikli kuyrukta da bu yaklaşımın avantajından söz edileceğini söyledi.

10. Alıştırma Problemleri

Hoca dersin sonunda üç algoritmik problem verdi. Bunları kodlayıp algoritmasını geliştirmenin hem gelecek haftaki öncelikli kuyruk konusuna hazırlık olacağını hem de konuyu ara sınav öncesinde çalışmış olmayı sağlayacağını söyledi. Problemler:

- Verilen bir ikili ağacın heap özelliği taşıyıp taşımadığını bulmak.
- Sırasız bir dizide k'ıncı en küçük ya da k'ıncı en büyük elemanı bulmak.
- Verilen bir min heap'i max heap'e dönüştürmek.

10.1 Bir İkili Ağaç Heap mi?

Hoca tahtaya birkaç ikili ağaç çizdi: heap özelliği sağlayan bir ağaç, son seviyesinde boşluk olduğu için heap olmayan bir ağaç ve 8, 9, 7, 6, 5 değerlerini içeren bir ağaç.

Hoca, heap olmamanın önce yapı bakımından, yani complete binary tree olup olmadığı açısından kontrol edilmesi gerektiğini söyledi. Ardından hem complete olma hem de max ya da min heap özelliği için birkaç dakikalık düşünme süresi verdi (01:54:35 - 02:00:00).

Soru: Bir ikili ağacın complete binary tree olup olmadığını nasıl kontrol ederiz? **Cevap:** Bir öğrenci, dizi gösteriminde 1'den n'e kadar geçersiz (boş) göz olmamasının kontrol edilebileceğini, ama bağlı yapıda bunu nasıl yapacağını bilemediğini söyledi. Hoca dizi kısmını doğruladı: Ağaç üzerinde bir diziyi gezer gibi gezerken atlanmış, arada boş kalmış bir eleman varsa arama durdurulur ve ağacın complete olmadığı söylenir. n'inci elemana kadar kesintiye uğramadan gidilirse ağaç complete'tir.

Bağlı yapıda (her ebeveynin çocuklarına işaretçi tuttuğu yapıda) nasıl ilerleneceği soru olarak bırakıldı. Hoca bu tür problemlerin hemen söylenince çok katkı sağlamayacağını, gelecek

haftaya kadar alıştırma olarak üzerinde çalışılmasını istedi. Vakit kalırsa öncelikli kuyruk dersinde cevaplarıyla ilgili açıklama yapacağını söyledi.

10.2 Min Heap'ten Max Heap'e

Hoca ipucu olarak bir min heap yazdı: `2, 3, 5, 9, 2, 8, 7`. Sonra "biraz daha farklı olsun" diyerek değerleri değiştirmeye başladı ve sordu: Bu min heap'i max heap'e dönüştürmek isteseniz ne yapardınız? Heap sort'ta ve silmede kullandığımız yaklaşımlardan yararlanabilir miyiz? Sorunun cevabı derste verilmedi.

Örnek Sorular

Bu ders için hazır kare ya da video yoktur. Sorular yalnız derste sözlü sorulan sorulardan kuruldu.

Soru 1 (derste sözlü, 00:10:46)

Min heap `2, 5, 7, 6, 8, 9, 11` ağacında 8 ile 11'in yeri değiştirilirse ağaç yine min heap olur mu?

Çözüm: Olur. 5'in çocukları 6 ve 11, 7'nin çocukları 9 ve 8 olur; ikisinde de ebeveyn çocuklarından küçüktür. Heap'te kardeşler arasında sıra kuralı yoktur.

Soru 2 (derste sözlü, 00:11:57)

Heapify işlemi ne anlama gelir? Hangi işlemlerden sonra gerekir?

Çözüm: Heapify, ağacı heap kurallarına (ebeveyn çocuklarından büyük ya da küçük) yeniden uydurma işlemidir. Ekleme ve silme heap yapısını bozabileceği için her ikisinden sonra yapılır. Bir diziden heap oluşturmak için de kullanılır.

Soru 3 (derste sözlü, 00:14:18)

Heap'e eleman ekleme ve heap'ten eleman silmenin karmaşıklığı nedir?

Çözüm: İkisi de en kötü durumda dahil $O(\log n)$'dir. Ağaç complete binary tree olduğu için dejenere olamaz; yüksekliği $\lfloor \log_2 n \rfloor$'dir.

Soru 4 (derste sözlü, 00:15:21)

`getMax` ve `getMin` işlemlerinin karmaşıklığı nedir?

Çözüm: Max heap'te `getMax`, min heap'te `getMin` $O(1)$'dir, çünkü aranan eleman köktedir.

Soru 5 (derste sözlü, 00:16:39)

Max heap'te en küçük eleman nerede olabilir?

Çözüm: Bir yaprakta. Dizi gösteriminde $\lfloor n/2 \rfloor + 1$ ile n arasındaki gözlere bakmak yeter.

Soru 6 (derste sözlü, 00:18:34)

Dizilerde ve bağlı listelerde hep kullandığımız hangi temel işlem heap işlemleri arasında yer almaz? Neden?

Çözüm: Arama (search). Heap'te elemanlar yalnız ebeveyn-çocuk sırasına göre dizilir; BST'deki gibi yön seçtiren bir kural yoktur. Bir değeri bulmak için bütün ağaç gezilir.

Soru 7 (derste sözlü, 00:25:24)

`maxHeapify` içinde sağ çocuk neden `A[i]` ile değil de `A[largest]` ile karşılaştırılır?

Çözüm: `largest` o ana kadar bulunan en büyük elemanın indeksidir; sol çocuk `A[i]`'den büyükse `largest` artık sol çocuktur. Sağ çocuk `A[i]` ile karşılaştırılsaydı, iki çocuk da `A[i]`'den büyükken büyük olan seçilemezdi.

Soru 8 (derste sözlü, 00:30:06)

Heapify işleminin en kötü durum karmaşıklığı nedir?

Çözüm: $O(\log n)$. Eleman her adımda bir seviye iner ve en fazla ağacın yüksekliği kadar iner.

Soru 9 (derste sözlü, 00:33:43)

Heap olup olmadığı bilinmeyen `5, 7, 6, 4, 8, 9, 1` ağacı max heap'e nasıl dönüştürülür? Heapify'a hangi düğümden başlanır?

Çözüm: Yaprak olmayan son düğümden, yani `n / 2 = 3` numaralı düğümden (değer 6) başlanır ve 1. düğüme kadar geriye doğru her düğüm için `maxHeapify` çağrılır. Ara sonuçlar: `5, 7, 9, 4, 8, 6, 1`, `5, 8, 9, 4, 7, 6, 1`, `9, 8, 6, 4, 7, 5, 1`. Sonuç `9, 8, 6, 4, 7, 5, 1`'dir (Bölüm 5.3).

Soru 10 (derste sözlü, 00:59:26)

`14, 7, 12, 13, 17, 6, 4, 9, 15` ağacında build max heap hangi iç düğümden başlar? Sonuç nedir?

Çözüm: $n = 9$, `n / 2 = 4`; 4. düğümden (değer 13) başlanır. Sonuç `17, 15, 12, 14, 7, 6, 4, 9, 13`'tür (Bölüm 5.4).

Soru 11 (derste sözlü, 01:18:30)

Heap'te silme yalnız max heap'te en büyükten, min heap'te en küçükten mi yapılır?

Çözüm: Genelde evet, çünkü amaç öncelikli kuyruktan en öncelikli elemanı çıkarmaktır. Ama herhangi bir eleman da silinebilir.

Soru 12 (derste sözlü, 01:22:26)

Max heap `19, 16, 18, 13, 12, 14, 15, 7, 6, 4, 5` heap yapısı kullanılarak nasıl sıralanır?

Çözüm: Kök son gözle yer değiştirir, `n` bir azaltılır, kökten heapify yapılır; bu tek eleman kalana kadar sürer. Sonuç `4, 5, 6, 7, 12, 13, 14, 15, 16, 18, 19`'dur. Tur tur iz Bölüm 9.5'tedir.

Soru 13 (derste sözlü, 01:35:39)

Heap sort'un karmaşıklığı nedir?

Çözüm: $O(n \log n)$. n turun her birinde $O(\log n)$'lik bir heapify yapılır.

Soru 14 (derste sözlü, 01:51:35)

Sırasız bir dizide k'ıncı en küçük (ya da en büyük) eleman heap ile nasıl bulunur?

Çözüm: Derste yalnız ipucu verildi: Heap sort'u tamamlamak yerine k tur yapmak, maliyeti $O(k \log n)$ düzeyine indirir.

> **Çözüm (derste verilmedi):** k'ıncı en küçük için dizi önce min heap'e dönüştürülür ($O(n)$). Sonra kök $k-1$ kez silinir; her silme $O(\log n)$'dir. Bu silmelerden sonra kökte k'ıncı en küçük eleman kalır. Toplam maliyet $O(n + k \log n)$'dir. k'ıncı en büyük için aynı işlem max heap ile yapılır. Soru 15'teki programda yer alan `kthSmallest` fonksiyonu bunu yapar; fonksiyon dersteki `5, 7, 6, 4, 8, 9, 1` dizisi üzerinde

$k = 1..7$ için ayrı bir sınama programıyla denendi ve her k için sıralı dizinin k'ıncı elemanını verdi.

Soru 15 (derste sözlü, 01:53:59)

Verilen bir ikili ağacın heap olup olmadığı nasıl bulunur? Ağaç dizi yerine bağlı yapıda (düğüm ve işaretçi) tutuluyorsa complete olup olmadığı nasıl kontrol edilir?

Çözüm: Derste dizi gösterimi için cevap verildi: 1'den n'e kadar arada boş göz olmamalıdır. Sonra sıra kuralı hem max hem min heap için kontrol edilir.

> **Çözüm (derste verilmedi):** Bağlı yapıda önce düğüm sayısı n sayılır. Kök 1 numaralı kabul edilir; `i` numaralı düğümün sol çocuğuna `2i`, sağ çocuğuna `2i + 1` numarası verilir. Ağaç complete ise numaralar tam olarak $1..n$ aralığını doldurur; bir düğümün numarası n'i geçiyorsa arada boşluk var demektir ve ağaç complete değildir. Ardından her düğüm çocuklarıyla karşılaştırılarak max ve min sıra kuralı kontrol edilir. Aşağıdaki program `gcc -Wall -Wextra` ile uyarısız derlendi. `heapType` 1 ise max heap, 2 ise min heap, 0 ise heap değildir. Dersteki `6, 4, 5, 3, 2, 0, 1` ağacı için 1 (max heap), aynı ağaçta son seviyede bir boşluk açılınca 0 döner.

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int key;
    struct node *left, *right;
};

struct node *newNode(int key) {
    struct node *p = malloc(sizeof(struct node));
    p->key = key;
    p->left = p->right = NULL;
    return p;
}

int countNodes(struct node *r) {
    if (r == NULL)
        return 0;
    return 1 + countNodes(r->left) + countNodes(r->right);
}

/* Kök 1 numaralı; i düğümünün çocukları 2i ve 2i+1.
   Hiçbir numara n'i geçmiyorsa ağaç complete'tir. */
int isComplete(struct node *r, int i, int n) {
    if (r == NULL)
        return 1;
    if (i > n)
        return 0;
    return isComplete(r->left, 2 * i, n) && isComplete(r->right, 2 * i + 1, n);
}

int isMaxOrder(struct node *r) {
    if (r == NULL)
        return 1;
    if (r->left && r->left->key > r->key)
        return 0;
    if (r->right && r->right->key > r->key)
        return 0;
    return isMaxOrder(r->left) && isMaxOrder(r->right);
}
```

```c
int isMinOrder(struct node *r) {
    if (r == NULL)
        return 1;
    if (r->left && r->left->key < r->key)
        return 0;
    if (r->right && r->right->key < r->key)
        return 0;
    return isMinOrder(r->left) && isMinOrder(r->right);
}

/* 1: max heap, 2: min heap, 3: ikisi de, 0: heap değil */
int heapType(struct node *r) {
    if (!isComplete(r, 1, countNodes(r)))
        return 0;
    return isMaxOrder(r) + 2 * isMinOrder(r);
}

void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}

void minHeapify(int A[], int i, int n) {
    int left = 2 * i, right = 2 * i + 1, smallest = i;
    if (left <= n && A[left] < A[smallest])
        smallest = left;
    if (right <= n && A[right] < A[smallest])
        smallest = right;
    if (smallest != i) {
        swap(&A[i], &A[smallest]);
        minHeapify(A, smallest, n);
    }
}

/* Sırasız A[1..n] içinde k'ıncı en küçük: min heap kur, k-1 kez kökü sil */
int kthSmallest(int A[], int n, int k) {
    for (int i = n / 2; i >= 1; i--)
        minHeapify(A, i, n);
    for (int j = 1; j < k; j++) {
        swap(&A[1], &A[n]);
        n--;
        minHeapify(A, 1, n);
    }
    return A[1];
}

int main(void) {
    /* 6, 4, 5, 3, 2, 0, 1 dizisinin bağlı gösterimi */
    struct node *a = newNode(6);
    a->left = newNode(4);
    a->right = newNode(5);
    a->left->left = newNode(3);
    a->left->right = newNode(2);
    a->right->left = newNode(0);
    a->right->right = newNode(1);
    printf("6,4,5,3,2,0,1 -> %d\n", heapType(a));

    /* sol çocuğu boş, sağ çocuğu dolu bir düğüm: complete değil */
    a->left->left = NULL;
```

```
    printf("bosluklu agac -> %d\n", heapType(a));
    return 0;
}
```

Çıktı:

```
6,4,5,3,2,0,1 -> 1
bosluklu agac -> 0
```

Soru 16 (derste sözlü, 02:03:46)

Verilen bir min heap max heap'e nasıl dönüştürülür? Heap sort'ta ve silmede kullanılan yaklaşımlardan yararlanılabilir mi?

> **Çözüm (derste verilmedi):** Dizinin min heap olması max heap kurmaya bir kolaylık sağlamaz; dizi sıradan bir dizi gibi `buildMaxHeap` ile işlenir. `n / 2`'den 1'e kadar her düğüm için `maxHeapify` çağrılır ve işlem $O(n)$'de biter. Altyazıda duyulan `2, 3, 5, 9, 2, 8, 7` dizisi üzerinde turlar şöyledir (python ile doğrulandı): `i = 3` (5): 8 yukarı, `2, 3, 8, 9, 2, 5, 7`; `i = 2` (3): 9 yukarı, `2, 9, 8, 3, 2, 5, 7`; `i = 1` (2): 9 yukarı, ardından 3 yukarı, `9, 3, 8, 2, 2, 5, 7`. Sonuç max heap'tir.

Dersin Toparlaması

Bu derste ikili yığın ağacının iki kuralı (complete binary tree olmak ve ebeveynin çocuklarından hep büyük ya da hep küçük olması) ile max heap ve min heap ayrımı görüldü. Heap'in dizide 1'den başlayan indekslerle (`2i`, `2i + 1`, `i / 2`) tutulduğu, en büyük ya da en küçük elemana $O(1)$'de erişildiği, ekleme ve silmenin en kötü durumda da $O(\log n)$ olduğu, aramaya ise uygun olmadığı vurgulandı. Ardından heap'in temel işlemi olan aşağı yönlü `maxHeapify` satır satır incelendi; rastgele bir dizinin son iç düğümden köke doğru heapify uygulanarak max heap'e dönüştürüldüğü iki örnekle gösterildi.

Dersin ikinci yarısında ekleme (sona koyup yukarı yönlü taşıma), silme (son elemanla yer değiştirip `n`'i azaltma ve heapify) ve bu ikisinin birleşimi olan heap sort işlendi. Heap sort $O(n \log n)$'dir ama kararlı değildir; yalnız ilk k eleman gerektiğinde $O(k \log n)$ ile avantaj sağlar. Hoca, heap olup olmama kontrolü, k'ıncı en küçük ya da en büyük eleman ve min heap'i max heap'e çevirme problemlerini alıştırma olarak bıraktı. Gelecek hafta bu yapının üzerine öncelikli kuyruklar (priority queue) anlatılacak.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.